Yongchun Li - Week 8 Report (*1378156*)

1. Tasks Accomplished This Week

Meetings & Collaboration:

1. Attended daily stand-up meetings (1 h).
2. Participated in the weekly team retro & next week planning(2 h), took part in the review & approval of a bunch of pull request backlogs, as well as decide the next steps & task allocation in accordance with the client's feedback and new requirements.
3. Participated in Login prototype, strategy discussion (20min).

Development Work: see PR#149(Updates applied since last report, closed Issue#128)

4. Implemented a scalable service path tracing system (12h):
   - ✓ Designed and built Transmittable Thread Local based cross-thread tracing mechanism
   - ✓ Developed request tracing architecture supporting multiple async execution models by adding context propagation support for various async patterns (thread pools, Completable Future, @Async, Reactor)
   - ✓ Implemented core components like TracingFilter.java with unique trace ID generation
   - ✓ Resolved complex thread model issues in Reactor Web Flux to ensure correct trace context propagation
   - ✓ Integrated trace IDs with database operation logs for end-to-end traceability
5. Responded to peer code reviews and take actions to solve the issue met. Such as :
   - ✓ shared code extraction into method in logging aspect
   - ✓ Thread pool config parameter into yaml

2. Key Issues Encountered

Initial tracing implementation using MDC (Mapped Diagnostic Context) failed in multi-threaded environments, as context was lost across async operations. Solved by implementing Alibaba's TransmittableThreadLocal (TTL) library, specifically designed for context propagation across thread pools and async invocations.

Reactor WebFlux environment presented significant challenges for trace ID propagation due to its publisher-subscriber model and unique thread scheduling mechanisms. After extensive experimentation, developed ReactorContextHook with custom Reactor Hooks and Schedulers integration. Implemented ThreadLocal safeguards to prevent recursive invocation stack overflow in operator chains.

Asynchronous log processing with trace IDs required careful thread management to prevent resource leaks. Initial implementation caused occasional task queue backups, resolved through proper thread pool configuration and shutdown procedures.

Database persistence for traced logs necessitated careful schema design considerations, particularly around indexing strategies to balance query performance with write throughput.

Other issues are mostly team-wide common, like lack official channel communication, lack frequency at client info synchronization.

3. Plan for Week 9

Sprint 3 starting:

- ✧ Collaborate with Weiran to optimize database schema based on new client requirements